

SUBJECT : PROBLEM SOLVING AND TESTING USING JAVA

STUDENT NAME : D.VAMSI

COURSE CODE : 10211CS227

VTU NUMBER : VTU27781

Task 2 (Difficult): Real-Time Stream Analytics Engine

Problem Description

A cloud analytics platform receives millions of sensor readings. Each reading contains a sensor ID and temperature value. Using stream processing concepts, perform the following operations:

1. Filter temperatures greater than 50.
2. Group readings by sensor ID.
3. Compute average temperature per sensor.
4. Sort sensors based on average temperature in descending order.

Input Format

- First line contains integer N.
- Next N lines contain SensorID and Temperature.

Output Format

Display SensorID and average temperature sorted in descending order.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq \text{Temperature} \leq 100$

Sample Input

6

S1 60

S2 40

S1 80

S3 70

S2 90

S3 30

Sample Output

S1 70.0

S2 90.0

S3 70.0

PROGRAM :

```
import java.util.*;
```

```
import java.util.stream.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        Map<String, List<Double>> data = new HashMap<>();
```

```
        for (int i = 0; i < n; i++) {
```

```
            String sensor = sc.next();
```

```
            double temp = sc.nextDouble();
```

```
            // Filter temperature > 50
```

```

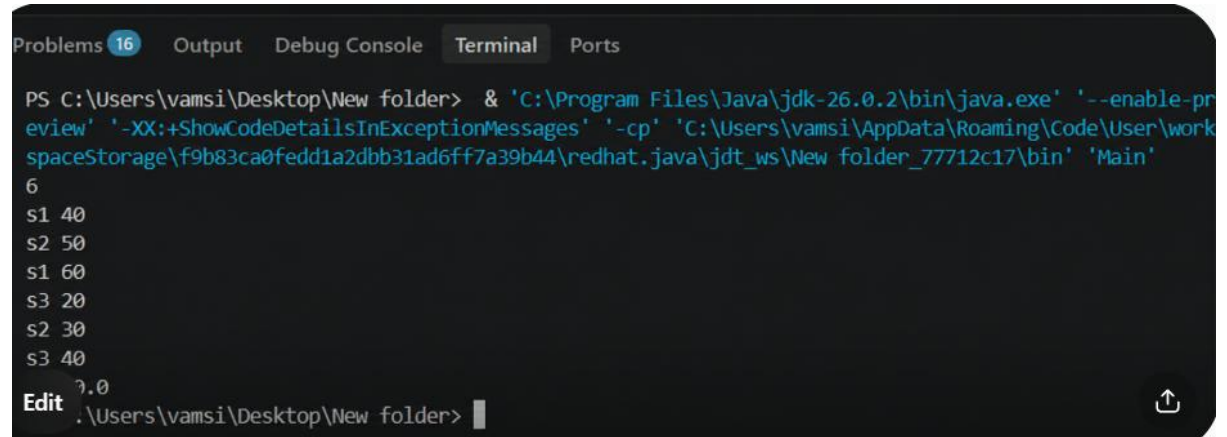
    if (temp > 50) {
        data.computeIfAbsent(sensor, k -> new ArrayList<>()).add(temp);
    }
}

// Calculate average and sort in descending order
data.entrySet().stream()
    .map(e -> new AbstractMap.SimpleEntry<>(
        e.getKey(),
        e.getValue().stream()
            .mapToDouble(Double::doubleValue)
            .average()
            .orElse(0)
    ))
    .sorted((a, b) -> Double.compare(b.getValue(), a.getValue()))
    .forEach(e -> System.out.println(
        e.getKey() + " " + e.getValue()
    ));

sc.close();
}
}

```

OUTPUT :



The screenshot shows an IDE's terminal window with the following content:

```
Problems 16 Output Debug Console Terminal Ports
PS C:\Users\vamsi\Desktop\New folder> & 'C:\Program Files\Java\jdk-26.0.2\bin\java.exe' '--enable-pr
eview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vamsi\AppData\Roaming\Code\User\work
spaceStorage\f9b83ca0fedd1a2dbb31ad6ff7a39b44\redhat.java\jdt_ws\New folder_77712c17\bin' 'Main'
6
s1 40
s2 50
s1 60
s3 20
s2 30
s3 40
Edit 7.0
.\Users\vamsi\Desktop\New folder>
```

The terminal output displays a series of values: 6, s1 40, s2 50, s1 60, s3 20, s2 30, and s3 40. The IDE interface includes tabs for Problems (16), Output, Debug Console, Terminal, and Ports. The command executed is a Java command with various flags and a classpath, running a class named 'Main'.